

Confiabilidade

Prof. Eduardo Santana

Introdução

- Confiabilidade indica a capacidade de um software funcionar corretamente e de forma consistente
- Pode ser medido de várias formas
 - Tempo que a aplicação está no ar -> Disponibilidade
 - Taxa de respostas com sucesso
- Influência todo o ciclo de desenvolvimento, manutenção e operação de software

Disponibilidade

- Métrica bastante importante, maioria das plataformas tem indicadores sobre sua disponibilidade

<https://www.githubstatus.com/>

<https://health.aws.amazon.com/health/status>

<https://status.botcity.dev/dashboard/7>

Disponibilidade

Disponibilidade	Indisponibilidade por ano	Indisponibilidade por mês
99% (two nines)	~3 dias 15h 39m	~7h 18m
99,9% (three nines)	~8h 45m	~43m
99,99% (four nines)	~52m	~4m 23s
99,999% (five nines)	~5m 15s	~26s
99,9999% (six nines)	~31s	~2,6s

Confiabilidade

- O que deve ser feito para aumentar a confiabilidade de uma aplicação?

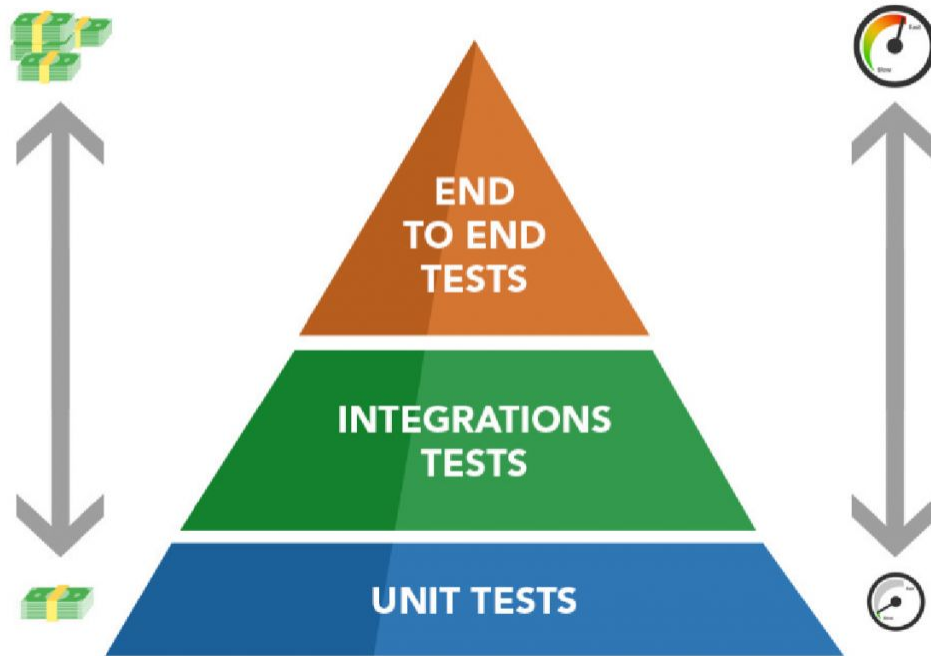
Confiabilidade

- O que deve ser feito para aumentar a confiabilidade de uma aplicação?
 - Testes
 - Tratamento de Erros
 - Integração Contínua (CI - Continuous Integration)
 - Entrega Contínua (CD - Continuous Delivery)
 - Automatização de Tarefas
 - Monitoramento
 - ...

Testes

- Verificar se o software funciona de acordo com o especificado
- Existem vários tipos de testes
- Testes manuais x testes automatizados
- Existem várias bibliotecas de teste
 - JUnit, Mockito, pytest, selenium,...

Testes de Software



Tratamento de Erros

- Toda linguagem possui mecanismos para o tratamento de erros
- Os mais comuns são os blocos de exceção, como o try/catch do Java
- Além disso, existem bibliotecas para validações, por exemplo, o spring-boot-starter-validation para a validação básica de dados de entrada

Tratamento de Erros

- O tratamento de erros é importante para usuários humanos
- Mas também é essencial para a comunicação máquina-máquina
- Por isso a importância de código de retorno corretos

Tratamento de Erros

https://pt.wikipedia.org/wiki/Lista_de_c%C3%B3digos_de_estado_HTTP

Integração Contínua (CI)

- Utilizar um repositório central que facilita a integração de código de uma forma simplificada
- Utilização do Git (ou outro repositório de código)
- Mas apenas usar o Git é insuficiente
 - Boas práticas e automatização de tarefas

Entrega Contínua (CD)

- Capacidade de fazer entregas de código o tempo todo de forma simples e automatizada
- Evitar o trabalho manual de fazer o deploy de uma aplicação
- Parar o processo caso algum problema seja encontrado

Integração Contínua e Entrega Contínua

- Em algumas aulas vamos implementar um pipeline de CI e CD usando o GitHub Actions
- Outras possíveis ferramentas são o Jenkins e o CircleCI
- Algumas tarefas que vamos automatizar:
 - Executar os testes automatizados quando um PR é aberto
 - Verificar se a aplicação tem o mínimo de cobertura de testes
 - Criar imagem Docker e subir no DockerHub
 - Notificar usuários que uma nova versão foi criada
 - Fazer o deploy automático da aplicação no servidor

Monitoramento

- Existem vários pontos de uma aplicação que precisam ser monitorados
- Aplicações, servidores, bancos de dados,...
- Existem várias ferramentas para isso:
 - Grafana, LogStash, DataDog,...